

<hackathon/>
Nicaragua 

KR



NOX 20
26

10 años ¡siempre más allá!

Guía paso a paso

Git y GitHub para equipos de desarrollo

*Control de versiones y colaboración tecnológica sin fricciones.
Todos los comandos del taller, explicados uno por uno.*

 8 de octubre de 2026

 3:15 – 5:15 p.m.

 CI Nicaragua



Speaker

Marvin Rafael Meza Pineda

Centro Nacional de Innovación y Tecnología
Francisco "El Chele" Moreno (CI Nicaragua)



Cómo usar esta guía

Esta guía acompaña el taller de principio a fin. Si te pierdes en algún momento, **busca el módulo en el que vas y sigue los pasos**: cada comando está explicado con **qué hace**, **cómo se escribe** y **qué deberías ver** en tu pantalla.

No importa si es tu primera vez en una terminal o si ya programas: los recuadros de “**Nivel extra**” tienen trucos para quienes quieran ir más allá.

Lo que significa cada recuadro



Concepto

La idea detrás del comando, en palabras simples.



Tip

Un atajo o consejo que te facilita la vida.



Cuidado

Errores comunes y cómo evitarlos.



Comprueba

Cómo saber que lo hiciste bien.



Nivel extra

Para quienes ya tienen experiencia.

Cómo leer los comandos

```
Ejemplo
$ git commit -m "mi mensaje"      # ← esto lo escribes tú
[main a1b2c3d] mi mensaje         # ← esto te responde Git
```

- El símbolo **\$** representa la terminal esperando una orden. **No lo escribas**: copia solo lo que va después.
- Lo que aparece en **azul** dentro de la terminal es la **respuesta** de Git. Puede variar un poco en tu computadora (por ejemplo, los códigos como **a1b2c3d** siempre son distintos).
- Lo que va después de **#** es un **comentario** para explicarte; no hace falta escribirlo.
- Las palabras entre **< >** se reemplazan por tus datos: **<rama>** → **experimento**. No escribas los símbolos **< >**.
- Después de escribir cada comando, presiona **Enter** ↵.

Contenido

0

Conceptos clave en 5 minutos



Preparación y terminal básica

1

Configura Git

2

Trae el repositorio del taller

3

Crea tu repositorio

4

Crea, modifica y guarda con commits

5 Ramas: crea y muévete entre ellas

6 .gitignore

 Reto final

 Solución de problemas y chuleta

Así se reparten las 2 horas

Hora	Bloque	Duración
3:15 p.m.	 Bienvenida y presentación del speaker	5 min
3:20 p.m.	 Introducción: ¿qué es Git y por qué usarlo?	15 min
3:35 p.m.	 Taller práctico: módulos 1 al 6	75 min
4:50 p.m.	 Preguntas, conclusiones y demo en vivo de push y Pull Request	15 min
5:05 p.m.	 Escaneo de asistencia para certificación y cierre	10 min

Conceptos clave en 5 minutos

1 ¿Qué es el control de versiones?

Es la **máquina del tiempo de tu código**. Registra cada cambio que haces, **quién** lo hizo, **cuándo** y **por qué**. Si algo se rompe, puedes volver a una versión anterior que sí funcionaba.



Piénsalo como un videojuego

Si pierdes en el nivel 5, no reinicias el juego desde cero: regresas al último **punto de control** guardado. En Git, cada punto de control se llama **commit**.

2 Git no es lo mismo que GitHub

Git

Herramienta local •  el motor

- Se instala en tu computadora.
- Funciona **sin internet**.
- Guarda la historia de tu proyecto.

GitHub

Plataforma en la nube •  el garaje

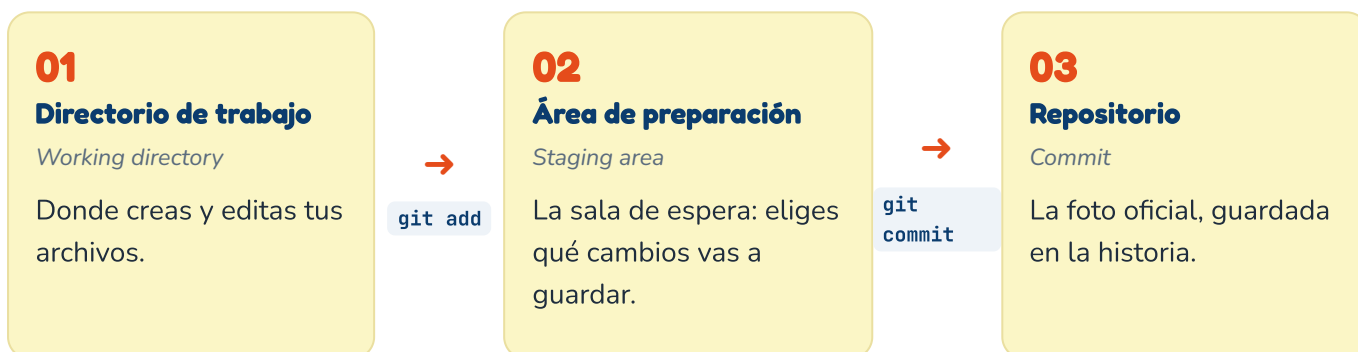
- Vive en internet.
- Guarda copias de tus repositorios.
- Permite compartir y colaborar en equipo.

3 Las palabras que más vas a escuchar

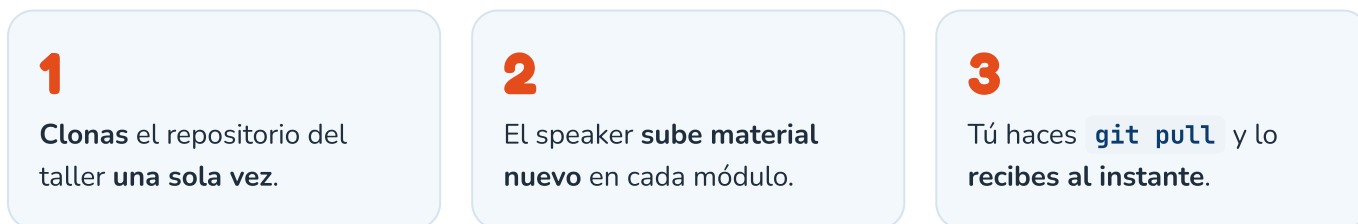
Palabra	Qué significa
Repositorio (repo)	La “carpeta inteligente” de tu proyecto: guarda tus archivos y toda su historia de cambios (en una carpeta oculta llamada <code>.git</code>).
Commit	Una “foto” de tu proyecto en un momento dado, con un mensaje que explica qué cambió.
Rama (branch)	Una línea de trabajo independiente. La principal se llama <code>main</code> .
Clonar	Descargar una copia completa de un repositorio que está en GitHub.
Pull	Traer a tu computadora los cambios nuevos de un repositorio remoto.
Push	Subir tus commits a GitHub. <i>Hoy solo lo verás en la demo final.</i>

4 Los 3 estados de un archivo

Git no guarda “todo a lo loco”. Tus cambios pasan por tres lugares:



5 Cómo trabajaremos hoy



Mientras tanto, practicas en **tu propio repositorio**, en una carpeta aparte:

 Escritorio/

-  NOMBRE-DEL-REPO/ ← **material del taller: aquí SOLO haces git pull**
-  mi-proyecto/ ← **tu práctica: commits, ramas y todo lo demás**



Regla de oro del taller

No hagas commits dentro de la carpeta del taller. Si tu copia y la del speaker se separan, el siguiente `git pull` fallará.

Preparación: tu equipo listo

1 Instala lo necesario

Programa	Para qué	Dónde descargarlo
Git	El programa de control de versiones. En Windows incluye Git Bash , la terminal que usaremos.	git-scm.com/downloads
Visual Studio Code	El editor donde abrirás tus archivos y la terminal.	code.visualstudio.com



No necesitas cuenta de GitHub

Para clonar un repositorio público y hacer `git pull` no hace falta iniciar sesión.

2 Abre la terminal correcta

1. Abre **Visual Studio Code**.
2. Ve al menú **Archivo** → **Abrir carpeta...** y elige tu **Escritorio**.
3. Abre la terminal con **Terminal** → **Nueva terminal** (o `Ctrl` + `ñ` / `Ctrl` + ```).
4. En **Windows**: en el panel de la terminal, haz clic en la flecha ▼ junto al botón + y elige **Git Bash**.



¿Por qué Git Bash?

Windows trae otras terminales (PowerShell y CMD) que usan comandos distintos. Con Git Bash, **todos los comandos de esta guía funcionan igual** en Windows, macOS y Linux.

3 Verifica que Git está instalado

`git --version`

Verificar

¿Qué hace? Le pregunta a Git qué versión tienes instalada. Si responde, Git está listo para usarse.

`git`

el programa

`--version`

"dime tu versión"

Git Bash

```
$ git --version
```

```
git version 2.47.0.windows.1 # tu número puede ser distinto
```



¿Dice “git: command not found” o “no se reconoce como comando”?

Git no está instalado o la terminal se abrió antes de instalarlo. Cierra VS Code por completo, vuelve a abrirlo y prueba de nuevo. Si sigue igual, pide ayuda al speaker.

Terminal básica: moverte sin miedo

Antes de usar Git, necesitas unos pocos comandos para moverte entre carpetas y crear archivos. Son los mismos en Git Bash, macOS y Linux.

Comando	Qué hace	Ejemplo
<code>pwd</code>	“¿Dónde estoy?” Muestra la ruta de la carpeta en la que estás.	<code>pwd</code>
<code>ls</code>	Lista los archivos y carpetas de donde estás.	<code>ls</code>
<code>ls -a</code>	Lista todo , incluso los archivos ocultos (los que empiezan con punto, como <code>.git</code>).	<code>ls -a</code>
<code>cd <carpeta></code>	Entra en una carpeta (<i>change directory</i>).	<code>cd mi-proyecto</code>
<code>cd ..</code>	Sale a la carpeta de arriba (los dos puntos significan “la carpeta padre”).	<code>cd ..</code>
<code>cd ~/Desktop</code>	Va directo a tu Escritorio (<code>~</code> es tu carpeta personal).	<code>cd ~/Desktop</code>
<code>mkdir <carpeta></code>	Crea una carpeta nueva (<i>make directory</i>).	<code>mkdir pruebas</code>
<code>touch <archivo></code>	Crea un archivo vacío .	<code>touch notas.txt</code>
<code>echo "texto" > archivo</code>	Escribe texto en un archivo. Reemplaza lo que tenía.	<code>echo "Hola" > a.txt</code>
<code>echo "texto" >> archivo</code>	Agrega una línea al final, sin borrar lo anterior.	<code>echo "Adiós" >> a.txt</code>
<code>cat <archivo></code>	Muestra el contenido de un archivo.	<code>cat a.txt</code>
<code>code <archivo></code>	Abre el archivo en VS Code .	<code>code index.html</code>
<code>clear</code>	Limpia la pantalla de la terminal (no borra nada).	<code>clear</code>

Practica: ¡pruébalo ahora!

```
$ pwd
/c/Users/tu-usuario/Desktop
$ mkdir pruebas && cd pruebas # && encadena dos comandos
$ echo "Hola" > saludo.txt
$ echo "Adiós" >> saludo.txt
$ cat saludo.txt
Hola
Adiós
$ cd ..
```



> vs >>

Un solo **>** **borra** el contenido y escribe encima. Dos **>>** **agregan** al final. Si dudas, usa **>>**.



Ahorra tiempo

Presiona **Tab** para autocompletar nombres de carpetas y archivos, y la **flecha ↑** para repetir comandos anteriores.



¿Tu Escritorio está en OneDrive?

Si **cd ~/Desktop** no funciona, prueba con **cd ~/OneDrive/Desktop** o **cd ~/OneDrive/Escritorio**. Con **pwd** siempre puedes confirmar dónde estás.

Módulo 1

Configura Git por primera vez

Antes de guardar cambios, Git necesita saber **quién eres**. Tu nombre y correo aparecerán en cada commit que hagas.



5 min



Se hace una sola vez por computadora



```
git config --global user.name "Tu Nombre"
```

Configurar

¿Qué hace? Guarda tu nombre para que Git lo use como autor de tus commits.

git

el programa

config

"configura algo"

--global

para todos tus proyectos en esta computadora

user.name

el dato: tu nombre

"Tu Nombre"

el valor, entre comillas

```
git config --global user.email "tu-correo@ejemplo.com"
```

Configurar

¿Qué hace? Guarda tu correo. Funciona igual que el anterior, pero con el dato **user.email**.

```
git config --global init.defaultBranch main
```

Configurar

¿Qué hace? Indica que la rama principal de tus repositorios nuevos se llame **main**, el nombre que hoy usa la industria (antes se llamaba **master**).

git config --list

[Revisar](#)

¿Qué hace? Muestra toda tu configuración para comprobar que quedó bien.

```
Git Bash

$ git config --global user.name "Ana López"
$ git config --global user.email "ana@ejemplo.com"
$ git config --global init.defaultBranch main
$ git config --list
...
user.name=Ana López
user.email=ana@ejemplo.com
init.defaultbranch=main
```



Comprueba

En la lista aparecen tu `user.name` y tu `user.email`. Si la lista es larga y no puedes escribir, presiona **q** para salir.



¿Te equivocaste?

Vuelve a escribir el mismo comando con el dato correcto: el nuevo valor reemplaza al anterior.

Módulo 2

Trae el repositorio del taller

Vas a descargar el repositorio del speaker con `git clone` y a mantenerlo actualizado con `git pull`. Así funciona un equipo: cada quien recibe el trabajo de los demás.



10 min

Clonar una vez y actualizar con pull

1 Ubícate en el Escritorio

```
Git Bash
$ cd ~/Desktop
$ pwd
/c/Users/tu-usuario/Desktop
```

2 Clona el repositorio

`git clone <url>`

Traer

¿Qué hace? Descarga una **copia completa** de un repositorio de GitHub: todos los archivos y toda su historia. Crea una carpeta nueva con el nombre del repositorio. **Se hace una sola vez.**

git

el programa

clone

“haz una copia”

<url>

la dirección del repositorio en GitHub

```
Git Bash
$ git clone https://github.com/TU-USUARIO/NOMBRE-DEL-REPO.git
Cloning into 'NOMBRE-DEL-REPO'...
remote: Enumerating objects: 15, done.
Receiving objects: 100% (15/15), done.
```



¿Dónde encuentro la URL?

En la página del repositorio en GitHub, botón verde <> **Code** → pestaña **HTTPS** → botón de copiar. En Git Bash se pega con **clic derecho** → **Paste** o con **Shift + Insert**.

3 Entra y mira lo que trajiste

```
Git Bash
$ cd NOMBRE-DEL-REPO
$ ls
Portada-Taller-Git.png  README.md
$ git log --oneline      # la historia que ya trae el repositorio
3c4d5e6 docs: agrega README del taller
a1b2c3d feat: estructura inicial
```

4 Actualiza con git pull

git pull

Traer

¿Qué hace? Revisa si hay cambios nuevos en GitHub y los **trae a tu copia**. Lo usarás al inicio de cada módulo para recibir el material que suba el speaker.

```
Si hay material nuevo

$ git pull
Updating 3c4d5e6..7f8a9b0
Fast-forward
 modulo-3.md | 12 ++++++++
 1 file changed, 12 insertions(+)
```

```
Si no hay nada nuevo

$ git pull
Already up to date.      # "ya estás al día"
```



clone vs pull

git clone copia el repositorio la primera vez. **git pull** lo actualiza cada vez que hay algo nuevo.



Recuerda la regla de oro

Esta carpeta es solo para recibir material. **No crees commits aquí**: tu práctica va en **mi-proyecto**.



Dos terminales = cero confusión

En VS Code, el botón + del panel de terminal abre otra terminal. Deja una en la carpeta del taller (para **git pull**) y otra en **mi-proyecto** (para practicar).

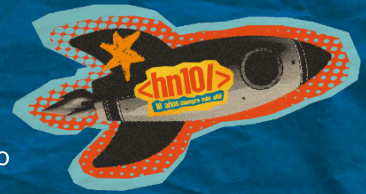
Módulo 3

Crea tu repositorio desde la terminal

Ahora crearás **tu propio** proyecto desde cero, en una carpeta aparte, y lo convertirás en un repositorio de Git.

🕒 10 min

🔗 mkdir + git init



Antes de empezar

En la terminal de la carpeta del taller, ejecuta **git pull** para recibir el material de este módulo.

1 Sal de la carpeta del taller y crea la tuya

```
Git Bash
$ cd ..           # sube al Escritorio
$ mkdir mi-proyecto # crea la carpeta
$ cd mi-proyecto  # entra en ella
```

2 Conviértela en repositorio

git init

Iniciar

¿Qué hace? Convierte la carpeta actual en un repositorio de Git. Crea adentro una carpeta oculta llamada **.git**, donde Git guardará **toda la historia** del proyecto.

git

el programa

init

“inicializa” (empieza a vigilar esta carpeta)

```
Git Bash
$ git init
Initialized empty Git repository in /c/Users/tu-usuario/Desktop/mi-proyecto/.git/
$ ls -a
./ ../ .git/           # ahí está la carpeta oculta
```



No borres la carpeta .git

Si la borras, pierdes toda la historia del proyecto (tus archivos quedan, pero Git “olvida” todo). Y nunca ejecutes `git init` dentro de la carpeta del taller: ya es un repositorio.



Pregúntale a Git cómo está todo

git status

[Revisar](#)

¿Qué hace? Es el comando **más útil** de Git: te dice en qué rama estás, qué archivos cambiaron y en qué estado está cada uno. Úsalo **todo el tiempo**; no modifica nada.

Git Bash

```
$ git status
```

```
On branch main           # estás en la rama main
```

```
No commits yet           # todavía no hay fotos guardadas
```

```
nothing to commit (create/copy files and use "git add" to track)
```



Comprueba

`git status` responde *On branch main* y *No commits yet*. Si dice *fatal: not a git repository*, no estás dentro de `mi-proyecto` o te saltaste `git init`.



¿Git te responde en español?

Según la instalación, Git puede mostrar sus mensajes traducidos (por ejemplo, *En la rama main*). Significan exactamente lo mismo.

Módulo 4

Crea, modifica y guarda con commits

El corazón de Git: crear archivos, prepararlos con `git add`, guardarlos con `git commit` y revisar la historia con `git log`.

🕒 20 min

🔗 add · commit · diff · log



Antes de empezar

Haz `git pull` en la terminal del taller. Luego vuelve a la terminal de `mi-proyecto`.

1

Crea archivos

```
Git Bash · mi-proyecto

$ touch index.html notas.txt           # dos archivos vacíos
$ echo "# Mi proyecto" > README.md     # un archivo con texto
$ ls
README.md  index.html  notas.txt
$ git status
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    README.md
    index.html
    notas.txt

nothing added to commit but untracked files present (use "git add" to track)
```



“Untracked” = sin seguimiento

Git ve los archivos (en **rojo**) pero todavía no los vigila. Están en el **directorio de trabajo**, el estado 1.

2 Prepara los cambios

`git add <archivo> · git add .`

Preparar

¿Qué hace? Mueve cambios al **área de preparación** (estado 2): le dices a Git “esto es lo que quiero guardar en la próxima foto”.

`git add README.md`

prepara solo ese archivo

`git add .`

el punto significa “todo lo de esta carpeta”

```
Git Bash · mi-proyecto

$ git add README.md
$ git status
Changes to be committed:
  new file:   README.md      # verde = preparado
Untracked files:
  index.html
  notas.txt
$ git add .                # ahora prepara todo lo demás
```

3 Guarda tu primer commit

`git commit -m "mensaje"`

Guardar

¿Qué hace? Toma todo lo que está en el área de preparación y lo guarda como una **foto permanente** en la historia (estado 3), junto con un mensaje que explica el cambio.

`git`

el programa

`commit`

“guarda la foto”

`-m`

“aquí va el mensaje”
(message)

`"..."`

el mensaje, siempre entre
comillas

```
Git Bash · mi-proyecto

$ git commit -m "feat: crea estructura inicial del proyecto"
[main (root-commit) a1b2c3d] feat: crea estructura inicial del proyecto
3 files changed, 1 insertion(+)
create mode 100644 README.md
create mode 100644 index.html
create mode 100644 notas.txt
$ git status
On branch main
nothing to commit, working tree clean    # todo guardado ✓
```



¿Olvidaste el -m y se abrió una pantalla rara?

Es el editor **Vim**. Para salir sin guardar: presiona **Esc**, escribe **:q!** y presiona **Enter**. Luego repite el commit con **-m "mensaje"**.

4

Modifica un archivo y mira qué cambió

```
Git Bash · mi-proyecto

$ echo "Primera idea del proyecto" >> notas.txt
$ cat notas.txt
Primera idea del proyecto
$ git status
Changes not staged for commit:
  modified:   notas.txt           # Git detectó el cambio
```

git diff

[Revisar](#)

¿Qué hace? Muestra **exactamente** qué líneas cambiaron desde el último commit. Las líneas con **+** (verde) se agregaron y las líneas con **-** (rojo) se eliminaron.

```
Git Bash · mi-proyecto

$ git diff
diff --git a/notas.txt b/notas.txt
--- a/notas.txt
+++ b/notas.txt
@@ -0,0 +1 @@
+Primera idea del proyecto
$ git add notas.txt
$ git commit -m "docs: agrega primera idea en notas"
[main e4f5a6b] docs: agrega primera idea en notas
1 file changed, 1 insertion(+)
```

5

Lista tus commits

git log · git log --oneline

[Historia](#)

¿Qué hace? Muestra la **historia** de commits, del más reciente al más antiguo. **git log** da todos los

detalles; `--oneline` muestra un commit por línea.

```
Git Bash · mi-proyecto

$ git log
commit e4f5a6b8c9d0e1f2a3b4c5d6e7f8a9b0c1d2e3f4 (HEAD → main)
Author: Ana López <ana@ejemplo.com>
Date: Thu Oct 8 15:52:10 2026 -0600

    docs: agrega primera idea en notas
...
$ git log --oneline
e4f5a6b (HEAD → main) docs: agrega primera idea en notas
a1b2c3d feat: crea estructura inicial del proyecto
```

e4f5a6b

el código único de cada
commit (*hash*)

HEAD → main

“estás aquí”, en la rama main



¿La terminal se quedó “trabada” con dos puntos (:) abajo?

Estás dentro del visor del historial. Usa las flechas para moverte y presiona **q** para salir.



Reto del módulo

Abre `index.html` con `code index.html`, escribe algo, guarda con **Ctrl + S** y crea un commit con ese cambio. Verifica que aparezca en `git log --oneline`.



Nivel extra

- `git add -p` : revisa y elige los cambios **pedazo por pedazo** antes de prepararlos.
- `git restore --staged <archivo>` : saca un archivo del área de preparación sin perder tus cambios.
- `git commit --amend -m "nuevo mensaje"` : corrige el mensaje del **último** commit (solo si aún no lo subiste a GitHub).
- `git log --stat` : muestra qué archivos tocó cada commit.

Módulo 5

Crea ramas y muévete entre ellas

Las ramas te permiten **experimentar sin miedo**: pruebas una idea en una línea de trabajo aparte mientras `main` sigue intacta.



20 min



branch · switch · log --graph



Antes de empezar

Haz `git pull` en la terminal del taller y vuelve a `mi-proyecto`.

1

Mira en qué rama estás

`git branch`

Ramas

¿Qué hace? Lista todas las ramas del repositorio. El asterisco `*` marca la rama en la que estás ahora.

```
Git Bash · mi-proyecto
$ git branch
* main
```

2

Crea una rama y cámbiate a ella

`git switch -c <rama>`

Ramas

¿Qué hace? Crea una rama nueva a partir de donde estás y te **mueve** a ella, todo en un solo paso.

git

el programa

switch

"cámbiate de rama"

-c

"créala primero" (create)

experimento

el nombre de la rama

```
Git Bash · mi-proyecto
$ git switch -c experimento
Switched to a new branch 'experimento'
$ git branch
* experimento
  main
```



Nombres de ramas

Usa minúsculas y guiones, sin espacios ni tildes: `experimento` , `nuevo-menu` , `feature/login` .

3

Trabaja en la rama

```
Git Bash · mi-proyecto (rama experimento)

$ echo "Probando un diseño nuevo" > diseno.txt
$ git add diseno.txt
$ git commit -m "feat: prueba nuevo diseño"
[experimento 9f8e7d6] feat: prueba nuevo diseño
1 file changed, 1 insertion(+)
$ ls
README.md  diseno.txt  index.html  notas.txt
```

4

Muévete entre ramas y observa la magia ✨

`git switch <rama>`

Ramas

¿Qué hace? Te cambia a una rama que ya existe. Tu carpeta se transforma para mostrar los archivos tal como están en esa rama.

```
Git Bash · mi-proyecto

$ git switch main
Switched to branch 'main'
$ ls
README.md  index.html  notas.txt
$ git switch experimento
Switched to branch 'experimento'
$ ls
README.md  diseno.txt  index.html  notas.txt
```

😞 ¡diseno.txt desapareció!

✨ ¡y volvió!



No se perdió nada

`diseno.txt` existe solo en la rama `experimento` . Cada rama guarda su propia versión del proyecto; por eso puedes probar ideas sin romper lo que ya funciona en `main` .



Guarda antes de cambiar de rama

Si tienes cambios sin commit, Git puede impedirte cambiar de rama para no perderlos. Haz un commit primero (o usa `git stash` , en el nivel extra).

5 Dibuja todas tus ramas

```
git log --oneline --graph --all
```

Historia

¿Qué hace? Muestra la historia de **todas** las ramas (`--all`) con un dibujo de líneas (`--graph`), un commit por renglón (`--oneline`).

```
Git Bash · mi-proyecto

$ git log --oneline --graph --all
* 9f8e7d6 (HEAD → experimento) feat: prueba nuevo diseño
* e4f5a6b (main) docs: agrega primera idea en notas
* a1b2c3d feat: crea estructura inicial del proyecto
```



Reto del módulo

Regresa a `main` , crea un commit ahí (por ejemplo, edita `README.md`) y vuelve a ejecutar `git log --oneline --graph --all` . ¡Verás cómo las dos ramas se separan en el dibujo!



Nivel extra

- `git merge experimento` (estando en `main`): une los cambios de la rama a `main` .
- `git branch -d experimento` : borra una rama que ya fue unida.
- `git stash` / `git stash pop` : guarda tus cambios en un cajón temporal para cambiar de rama y luego los recupera.
- `git checkout <rama>` es la forma antigua de `git switch` ; la verás en muchos tutoriales.

Módulo 6

.gitignore: lo que Git no debe guardar

Aprende a proteger contraseñas, claves y archivos temporales para que **nunca** entren en la historia del proyecto.

🕒 10 min

🔒 Proteger lo sensible



¿Qué es .gitignore?

Es un archivo de texto con una **lista de archivos y carpetas que Git debe ignorar**. Todo lo que esté en esa lista no aparece en `git status` y no se guarda en ningún commit.

1

Crea un archivo “secreto”

```
Git Bash · mi-proyecto

$ git switch main
$ echo "PASSWORD=123456" > secretos.env
$ git status
Untracked files:
  secretos.env      # 🧠 Git quiere guardarlo
```

2

Crea el .gitignore

```
Git Bash · mi-proyecto

$ echo "secretos.env" > .gitignore      # ignora ese archivo
$ echo "*.log" >> .gitignore            # y todos los .log
$ cat .gitignore
secretos.env
*.log
$ git status
Untracked files:
  .gitignore      # 🧠 secretos.env ya no aparece
```

3 Guarda el .gitignore en la historia

```
Git Bash · mi-proyecto

$ git add .gitignore
$ git commit -m "chore: agrega .gitignore"
[main 2b3c4d5] chore: agrega .gitignore
1 file changed, 2 insertions(+)
```

4 Cómo se escriben las reglas

Regla	Qué ignora
<code>secretos.env</code>	Ese archivo exacto.
<code>*.log</code>	Todos los archivos que terminan en <code>.log</code> (el <code>*</code> significa "cualquier nombre").
<code>venv/</code>	Toda la carpeta <code>venv</code> y lo que tiene adentro.
<code># comentario</code>	Nada: las líneas con <code>#</code> son notas para las personas.

Ejemplo para un proyecto real

```
# Credenciales
.env
*.key

# Entornos y archivos generados
venv/
__pycache__/
node_modules/

# Temporales
*.log
```



.gitignore sí se guarda

El archivo `.gitignore` se incluye en el repositorio para que **todo el equipo** ignore los mismos archivos.



Llegar tarde no sirve

`.gitignore` solo ignora archivos que **todavía no** están en un commit. Agrégalo al inicio del proyecto.



Nivel extra

Si ya guardaste un archivo por error, `git rm --cached <archivo>` lo saca del seguimiento sin borrarlo de tu computadora. Ojo: sigue existiendo en los commits anteriores; si era una contraseña real, cámbiala.



Reto final

Demuestra lo que aprendiste. Hazlo desde la terminal y, si puedes, **sin mirar la guía**. Marca cada paso cuando lo termines:

- ☐ Regresa al Escritorio con `cd ..`, crea la carpeta `mi-portafolio` y conviértela en repositorio.
- ☐ Crea `README.md` y `sobre-mi.txt`, y guárdalos en un commit.
- ☐ Agrega una línea a `sobre-mi.txt` y guarda el cambio en un segundo commit.
- ☐ Revisa con `git diff` antes de preparar el cambio.
- ☐ Crea la rama `proyectos`, agrega `proyectos.txt` y haz un commit.
- ☐ Regresa a `main` y comprueba con `ls` que `proyectos.txt` no está.
- ☐ Agrega un `.gitignore` que ignore los archivos `.env` y guárdalo en un commit.
- ☐ Muestra tu historia con `git log --oneline --graph --all`. 🎉

```
Solución (¡intenta primero sin verla!)

$ cd .. && mkdir mi-portafolio && cd mi-portafolio && git init
$ echo "# Mi portafolio" > README.md
$ echo "Hola, soy estudiante de programación" > sobre-mi.txt
$ git add . && git commit -m "feat: crea portafolio"
$ echo "Me gusta Python" >> sobre-mi.txt
$ git diff
$ git add sobre-mi.txt && git commit -m "docs: amplía sobre mí"
$ git switch -c proyectos
$ echo "Proyecto 1: calculadora" > proyectos.txt
$ git add . && git commit -m "feat: agrega lista de proyectos"
$ git switch main && ls
$ echo "*.env" > .gitignore
$ git add .gitignore && git commit -m "chore: agrega .gitignore"
$ git log --oneline --graph --all
```

★ Bonus para quienes ya tienen cuenta de GitHub

¿Terminaste antes? En el README del repositorio está el reto “**Colabora con un Pull Request**”: haz un **fork**, agrega tu nombre a la lista de participantes y propón el cambio. No necesitas permisos en el repositorio del speaker.



Buenas prácticas para llevarte

✓ Hazlo así

`feat: agrega validación de formulario`

Commits pequeños y frecuentes

✗ Evita esto

`cambios varios`

Un commit gigante al final del día

✓ Hazlo así	✗ Evita esto
<code>.gitignore</code> para archivos sensibles	Guardar contraseñas o archivos <code>.env</code>
Experimentar en una rama	Probar ideas arriesgadas directamente en <code>main</code>
Revisar con <code>git status</code> antes de cada commit	Usar <code>git add .</code> sin mirar qué se guarda

Prefijos para tus mensajes de commit

Prefijo	Úsalo cuando...	Ejemplo
<code>feat:</code>	Agregas una funcionalidad ✨	<code>feat: agrega inicio de sesión</code>
<code>fix:</code>	Corriges un error 🐛	<code>fix: corrige cálculo del total</code>
<code>docs:</code>	Cambias documentación 📄	<code>docs: actualiza README</code>
<code>chore:</code>	Haces tareas de mantenimiento 🛠	<code>chore: agrega .gitignore</code>



Si algo sale mal

Los errores son normales y casi siempre tienen una solución sencilla. Busca el mensaje que ves en tu pantalla:

Si ves esto...	Significa	Solución
<code>git: command not found</code> o “no se reconoce como comando”	Git no está instalado o la terminal se abrió antes de instalarlo.	Reinicia VS Code. Si sigue igual, reinstala Git.
<code>fatal: not a git repository</code>	No estás dentro de un repositorio.	Usa <code>pwd</code> y <code>cd</code> para ir a <code>mi-proyecto</code> , o ejecuta <code>git init</code> .
<code>Author identity unknown / Please tell me who you are</code>	Git no sabe tu nombre ni tu correo.	Repite el Módulo 1.
<code>nothing to commit, working tree clean</code>	¡No es un error! No hay cambios nuevos que guardar.	Modifica algo y vuelve a intentarlo.
<code>nothing added to commit but untracked files present</code>	Olvidaste preparar los archivos.	Ejecuta <code>git add .</code> antes del commit.
<code>pathspec '...' did not match any files</code>	El nombre del archivo está mal escrito o no estás en la carpeta correcta.	Revisa con <code>ls</code> y usa <code>Tab</code> para autocompletar.
Pantalla con <code>:</code> abajo que no responde	Estás en el visor de <code>git log</code> .	Presiona <code>q</code> .
Pantalla llena de <code>~</code> después de <code>git commit</code>	Se abrió el editor Vim porque faltó <code>-m</code> .	<code>Esc</code> , escribe <code>:q!</code> y presiona <code>Enter</code> .

Si ves esto...	Significa	Solución
<code>LF will be replaced by CRLF</code>	Aviso normal de Windows sobre los saltos de línea.	Puedes ignorarlo.
<code>fatal: destination path '...' already exists</code>	Ya clonaste ese repositorio en esta carpeta.	Entra con <code>cd</code> y usa <code>git pull</code> .
<code>Your local changes would be overwritten</code>	Tienes cambios sin guardar que chocan con la rama o con el <code>pull</code> .	Haz un commit primero (o pide ayuda).
<code>divergent branches</code> o <code>Need to specify how to reconcile</code> al hacer <code>git pull</code>	Hiciste commits dentro de la carpeta del taller y tu copia se separó.	Avisa al speaker. Para la próxima, practica solo en <code>mi-proyecto</code> .



Tu mejor amigo cuando estás perdido

`git status` casi siempre te dice qué está pasando y qué comando usar a continuación. Léelo con calma.



Chuleta de comandos

Todos los comandos del taller en una sola página. ¡Tenla a mano!

Para...	Comando	Qué hace
Terminal	<code>pwd</code>	Muestra dónde estás
	<code>ls</code> / <code>ls -a</code>	Lista archivos (con <code>-a</code> , también los ocultos)
	<code>cd <carpeta></code> / <code>cd ..</code>	Entra en una carpeta / sube un nivel
	<code>mkdir <carpeta></code>	Crea una carpeta
	<code>touch <archivo></code>	Crea un archivo vacío
	<code>echo "texto" >> <archivo></code>	Agrega texto al final de un archivo
	<code>cat <archivo></code>	Muestra el contenido de un archivo
Configurar	<code>git config --global user.name "..."</code>	Define tu nombre
	<code>git config --global user.email "..."</code>	Define tu correo
Traer	<code>git clone <url></code>	Copia un repositorio (una vez)
	<code>git pull</code>	Trae los cambios nuevos
Iniciar	<code>git init</code>	Convierte la carpeta en repositorio
Revisar	<code>git status</code>	Estado de los archivos
	<code>git diff</code>	Qué líneas cambiaron
	<code>git log --oneline</code>	Historia resumida
Preparar	<code>git add <archivo></code> / <code>git add .</code>	Prepara uno o todos los cambios
Guardar	<code>git commit -m "mensaje"</code>	Guarda la foto en la historia
Ramas	<code>git branch</code>	Lista las ramas (* = la actual)
	<code>git switch -c <rama></code>	Crea una rama y entra en ella
	<code>git switch <rama></code>	Cambia de rama
	<code>git log --oneline --graph --all</code>	Dibuja todas las ramas
Ignorar	<code>.gitignore</code>	Lista de archivos que Git no guardará



Para seguir aprendiendo

Pro Git (en español)

`git-scm.com/book/es/v2`
El libro oficial de Git, gratis.



Learn Git Branching

`learngitbranching.js.org`
Aprende ramas jugando, de forma visual.



GitHub Skills

`skills.github.com`
El siguiente paso: subir proyectos y colaborar.



Documentación de GitHub

`docs.github.com/es`
Guías oficiales en español.

¡Ahora, a versionar!



Ya sabes proteger, organizar y experimentar con tu código como lo hacen los equipos profesionales.

Marvin Rafael Meza Pineda

Centro Nacional de Innovación y Tecnología Francisco “El Chele” Moreno

★ Si este taller te ayudó, ¡dale una estrella al repositorio!

